

turing_service · Official Integration Guide

This guide is for developers integrating **your** system with turing, the voice-calling micro-service that places outbound AI voice campaigns ("batches") on your behalf and reports outcomes back to you.

Scope: this guide covers **registration, authentication, agents, single calls, batches, analytics, and self-serve management** · everything you need from onboarding to production.

1. Concepts

| Term | Meaning | |---|---| | **Client** | Your organization, as registered with turing. Identified by a `client_id` (UUID). | | **API key** | The credential your backend sends on every request (X-API-Key). Yours alone · you never see another client's data. | | **Agent** | A configured voice-calling persona. turing exposes only the agents an admin has **enabled** for your client. | | **Batch** | A campaign: one agent, a list of recipients, placed as a set of outbound calls. | | **Execution** | One call within a batch. |

Everything you create or read through turing is scoped to your `client_id`. You cannot see, list, or modify another client's batches · referencing one `batch_id` returns `404`, not an error revealing that it exists. Agents work a bit differently: any `agent_id` that isn't enabled for you (whether it belongs to another client, doesn't exist, or simply hasn't been turned on for you yet) returns `403 agent_not_enabled`, not `404` · see §5 and §6.

2. Base URL

`https://turing.docstribе.com/v1`

All endpoints below are relative to this base except registration, which is also under `/v1` (see below), and the outcome webhook, which is not under `/v1` at all.

The full auto-generated OpenAPI schema is browsable, unauthenticated, at `https://turing.docstribе.com/docs` (Swagger UI) and `https://turing.docstribе.com/openapi.json` · use it to cross-check exact field types if anything here seems ambiguous.

3. Onboarding · getting your API key

turing uses self-serve registration followed by admin approval; you cannot mint your own key.

1 Register:

```
curl -X POST https://turing.docstribе.com/v1/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Your Company", "contact_email":
    "eng@yourcompany.com"}'
```

| Field | Required | Notes | |---|---|---| | `name` | yes | 1-128 characters | | `contact_email` | no | Optional (up to 256 characters). Useful for operator follow-up. |

Response (201):

```
{ "status": "pending", "message": "Registration received
  · an operator will review your request." }
```

Registering twice with the same `name` returns the same pending record rather than erroring · safe to retry. By default, at most 10 registration attempts per source IP are accepted per hour (429 `rate_limited` beyond that).

- 2 **Wait for approval.** An operator on turing's side reviews and approves your registration out-of-band. On approval, an API key is generated and given to you **once** (over a secure channel · it is never retrievable again; if lost, ask for a new one to be issued).
- 3 **Store the key securely.** Treat it like any bearer credential:
 - Send it only over HTTPS, only in the X-API-Key header · never in URL query string.
 - Keep it server-side. Do not embed it in mobile or browser code.
 - It is not logged or displayed by turing after issuance.

Until approved, no key exists and every `/v1/*` call you make will fail authentication.

Claim links · one-click onboarding

If turing's operator shares a claim link instead of a manual handoff, the flow is:

- 1 **Peek** (safe for URL previews / unfurlers):

```
curl https://turing.docstribе.com/v1/claim/{token}

{ "client_name": "Your Company", "expires_in_seconds":
3540 }
```

Returns the client name and remaining TTL. Does **not** consume the link.

- 2 **Burn** (one-time · the link is destroyed on success):

```
curl -X POST
https://turing.docstribе.com/v1/claim/{token}

{ "client_name": "Your Company", "api_key": "tk_xxxx..."
}
```

Returns the raw API key **once**. Store it immediately · the token is deleted and cannot be used again.

Both endpoints are unauthenticated. If the token is expired, already used, or invalid, you get 404 · the response is identical regardless of the reason, so there is no way to enumerate valid tokens.

Portal lookup · recover access

If you've lost your API key but know your registered name and email:

```
curl -X POST https://turing.docstribе.com/v1/portal/lookup \
-H "Content-Type: application/json" \
-d '{"name": "Your Company", "email": "eng@yourcompany.com"}'

{ "key_id": "uuid", "api_key": "tk_xxxx..." }
```

Both name and email must match (case-insensitive) and your client must be **active**. If either doesn't match, or your client is pending/suspended, you get 404 · intentionally generic. Rate-limited per source IP.

4. Authentication

Every request (other than registration) must carry:

```
X-API-Key: tk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

| Situation | Response | |---|---| | Header missing | 401 unauthorized | | Key invalid/unknown | 401 unauthorized | | Key valid, but your client is not active (pending, suspended, or rejected) | 403 forbidden ("Client is not active") |

There is no session or token expiry to manage · every request is authenticated independently. If your key is ever compromised, ask turing's operator to revoke it and issue a new one.

5. Error format

Every error response · regardless of endpoint · has the same envelope:

```
{
  "error": "missing_required_variables",
  "message": "Human-readable summary",
  "detail": { "...": "endpoint-specific structured detail, or null"
},
  "request_id": "5286204c5e0747159fd1d83d74d8b2b2"
}
```

Include `request_id` when reporting an issue · it correlates to turing's server-side logs.

Common error codes you'll encounter:

| HTTP | error | Meaning | |---|---|---| | 401 | unauthorized | Missing/invalid X-API-Key | | 403 | forbidden | Client suspended | | 403 | agent_not_enabled | The `agent_id` you referenced isn't enabled for your client · this applies whether the agent belongs to another client, doesn't exist, or just hasn't been enabled for you yet. Agents never 404, always this. | | 404 | not_found | No such **batch** for your client (may exist for another client · you can tell) | | 422 | missing_required_variables | Recipient(s) missing a variable the agent's prompt requires | | 422 | validation_error | Malformed request body | | 429 | rate_limited | Only applies to `/v1/register` | | 404/405 | http_error | Generic Starlette/FastAPI fallback · e.g. a typo path or wrong HTTP method that doesn't match any route | | 502 or passthrough | voice_engine_error | The upstream voice engine failed or was unreachable. 502 when the engine couldn't be reached at all; when it *responded* with its own error, that status code (e.g. 400, 500) is passed through unchanged instead. | | 500 | internal_error | Unexpected server error |

6. Agents · discover what you can call with

Agents are configured and enabled for your client by turing's operator. You only ever see agents that have been explicitly enabled for you.

GET `/v1/agents`

```
curl https://turing.docstribе.com/v1/agents -H "X-API-Key: $KEY"
```

```
[
  {
    "id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
    "agent_name": "Booking Agent",
    "agent_status": "processed",
    "display_name": null
  }
]
```

Use `id` as the `agent_id` in every batch request below. If an agent you expect isn't listed, it isn't enabled for you yet (or was removed upstream · ask the operator).

GET /v1/agents/{agent_id}/variables

Before sending recipients, check what data the agent's prompt actually needs · every **required** variable **must** be present per recipient, or your batch is rejected before anything is placed.

```
curl https://turing.docstribе.com/v1/agents/37768781.../variables
-H "X-API-Key: $KEY"
```

```
{
  "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
  "required": ["patient_name", "doctors_name", "Follow_Up_Date"],
  "optional": ["Personalised_Symptoms"],
  "system_injected": ["agent_id", "call_sid", "current_date",
    "current_time",
    "execution_id", "from_number", "timezone",
    "to_number"],
  "all_prompt_variables": ["...union of required + optional..."]
}
```

- **required** · must be a key on every recipient object you send.
- **optional** · may be omitted per-recipient.
- **system_injected** · turing/the voice engine supplies these automatically, don't send them yourself.

403 `agent_not_enabled` here means the same as elsewhere: this agent isn't yours to use.

GET /v1/agents/drift · detect removed agents

Lists agents that were enabled for your client but have since disappeared from the voice engine (renamed, deleted, or unconfigured upstream).

```
curl https://turing.docstribе.com/v1/agents/drift -H "X-API-Key:
$KEY"
```

```
[
  {
    "id": "...",
    "voice_agent_id": "37768781-...",
    "event_type": "agent_missing",
    "detail": "Agent no longer found in voice engine catalog",
    "acknowledged": false,
    "created_at": "2026-08-15T..."
  }
]
```

Use this to detect if an agent you rely on has been removed or renamed. An empty list means all your agents are healthy.

7. Phone numbers

GET /v1/phone-numbers

```
curl https://turing.docstribе.com/v1/phone-numbers -H "X-API-Key: $KEY"
```

```
{
  "default_from_number": "+91XXXXXXXXXX",
  "phone_numbers": [
    { "phone_number": "+91XXXXXXXXXX" }
  ]
}
```

`default_from_number` is what turing uses as caller ID if you omit `from_phone_numbers` on a batch · either your client's configured default, or the service-wide fallback. You rarely need to set this explicitly.

There is currently no self-service endpoint for you to **set** your own `default_from_number` or `webhook_url` (§11) · these are configured for you by turing's operator on request. If you need either changed, ask them. You **can** read your current config (including whether a webhook secret is set) via `GET /v1/me/config` (§13).

8. Single calls

For one-off calls (not part of a batch campaign), use the calls endpoint directly.

POST /v1/calls · make a single call

```
curl -X POST https://turing.docstribе.com/v1/calls \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
  -d '{
    "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
    "recipient_phone_number": "+919876543210",
    "user_data": {
      "patient_name": "Asha Rao",
      "doctors_name": "Dr. Mehta",
      "Follow_Up_Date": "2026-07-20"
    }
  }'
```

Request fields:

Field	Required	Notes
<code>agent_id</code>	yes	Must be enabled for your client
<code>recipient_phone_number</code>	yes	E.164 format
<code>from_phone_number</code>	no	Caller ID override. Omit to use default.
<code>user_data</code>	no	Object of prompt variables for this call
<code>scheduled_at</code>	no	ISO 8601 for scheduling
<code>retry_config</code>	no	Same structure as batch retry config
<code>bypass_call_guardrails</code>	no	Skip voice engine calling-time guardrail checks (e.g. do-not-call windows). Default <code>false</code> .

Response (201):

```
{
  "execution_id": "exec_xyz",
  "status": "queued",
  "message": "Call placed successfully",
  "warnings": []
}
```

`execution_id` may be null if the voice engine accepted the request but hasn't assigned an execution id yet (e.g. scheduled calls). Poll GET `/v1/calls` to pick it up once it's available.

GET `/v1/calls` · list calls

```
curl "https://turing.docstribе.com/v1/calls?page=1&page_size=50" \
-H "X-API-Key: $KEY"
```

Query params: `agent_id`, `batch_id`, `status`, `outcome`, `urgency`, `q`, `date_from`, `date_to`, `page`, `page_size`.

GET `/v1/calls/{execution_id}` · call details

Returns full call record including transcript, analysis, extracted data, recording URL, cost, duration, `patient_ref` (if set), `retry_count`, and `error_message` (if the call failed).

POST `/v1/calls/{execution_id}/stop` · stop in-flight call

POST `/v1/calls/{execution_id}/analyze` · run LLM analysis

Triggers (or re-runs) LLM-based analysis on a completed call. Returns outcome classification, summary, urgency, confidence, and extracted data.

9. Batches · placing a campaign

A batch is one agent called against a list of recipients. There are two ways to create one: a **JSON list** (recommended · turing converts it to CSV for you) or a **raw CSV upload**.

9.1 POST `/v1/batches` · create from JSON

```
curl -X POST https://turing.docstribе.com/v1/batches \
-H "X-API-Key: $KEY" -H "Content-Type: application/json" \
-d '{
  "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
  "recipients": [
    {
      "contact_number": "+919876543210",
      "patient_name": "Asha Rao",
      "doctors_name": "Dr. Mehta",
      "Follow_Up_Date": "2026-07-20"
    }
  ],
  "from_phone_numbers": null,
  "retry_config": {
    "enabled": true,
    "max_retries": 2,
    "retry_on_statuses": ["no-answer", "busy"],
    "retry_intervals_minutes": [15, 60]
  },
  "webhook_url": null
}'
```

Request fields:

| Field | Required | Notes | |---|---|---| | `agent_id` | yes | Must be enabled for your client (see §6) | `recipients` | yes | Non-empty list of objects. Each **must** include `contact_number` (E.164). Any other key is a prompt variable for that call. | | `from_phone_numbers` | no | Pool of caller IDs (E.164). Omit to use your configured/default number. | | `retry_config` | no | Auto-retry behavior for failed calls (all sub-fields optional: `enabled`, `max_retries`, `retry_on_statuses`, `retry_on_voicemail`, `retry_intervals_minutes`). Each value in `retry_intervals_minutes` **must be** `15` this is the voice engine's own minimum. Turing validates it and returns `422 validation_error` up front if violated, so you never get a passthrough `voice_engine_error` for this specific case. | | `webhook_url` | no | **Leave unset unless you specifically need this.** See warning below. |

`webhook_url` replaces Turing as the callback target · it does not add a second destination. By default Turing tells the voice engine to send execution updates to its own receiver (`/webhooks/voice`), which is how Turing populates `GET .../executions`, keeps its own records current, and forwards you the signed outcome described in §11. If you set `webhook_url` yourself, the voice engine sends its **raw, unsigned** execution payload **directly to your URL instead** · Turing never receives it for that batch. Concretely, if you set this: you will *not* get Turing's signed §11 push for this batch's calls, and Turing's own DB won't reflect new statuses until you next call `GET /v1/batches/{batch_id}/executions` (which still works · it polls the voice engine directly, independent of any webhook). Only set this field if you specifically want the voice engine's raw callback instead of Turing's normalized, signed one.

Validation (before anything is placed): every recipient is checked against the agent's variable contract. If any recipient is missing a `required` variable, the **entire batch is rejected** with `422` and no calls are placed:

```
{
  "error": "missing_required_variables",
  "message": "missing_required_variables",
  "detail": {
    "error": "missing_required_variables",
    "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
    "required": ["patient_name", "doctors_name", "Follow_Up_Date"],
    "optional": [],
    "rows": [{"row": 3, "missing": ["Follow_Up_Date"]}]}
},
  "request_id": "..."
}
```

(`detail` mirrors the full structured error Turing raised internally, hence the repeated `error` key · read `error/message` at the top level, and `detail.rows` for exactly which recipients failed and why.)

Fix the flagged rows and resend. You can opt out of this check with `?validate=false` on the request, but then a malformed recipient fails silently on the voice engine's side instead of being caught up front · **not recommended**.

Success response (201):

```
{
  "batch_id": "b_abc123",
  "state": "created",
  "warnings": ["variable 'nickname' was sent but the agent's prompt does not use it"]
}
```

`warnings` are non-blocking · extra fields you sent that the agent's prompt doesn't reference. `batch_id` is what you use for every endpoint below.

No idempotency key. There is currently no dedup mechanism on batch creation · if your request times out or your connection drops after the voice engine already accepted it, blindly retrying `POST /v1/batches` will create a **second real batch** (and place a second round of real calls). Before retrying an ambiguous failure, check `GET /v1/batches/by-agent/{agent_id}` for a batch you already recognize, or maintain your own client-side ledger of submitted requests.

9.2 POST /v1/batches/upload · create from a CSV file

Same effect as above, but you supply a CSV directly instead of a JSON list · useful for very large recipient lists.
multipart/form-data:

```
curl -X POST https://turing.docstribе.com/v1/batches/upload \
  -H "X-API-Key: $KEY" \
  -F "agent_id=37768781-4fc4-4df2-a80f-a847b6dad8d2" \
  -F "file=@recipients.csv" \
  -F 'from_phone_numbers=["+91XXXXXXXXXX"]'
```

Form fields (multipart/form-data):

Field	Required	Notes
agent_id	yes	Must be enabled for your client
file	yes	CSV file, UTF-8, comma-delimited, header row required
from_phone_numbers	no	JSON array string, e.g. ["+91..."]
webhook_url	no	Same replaces-turing behavior described in §9.1 · leave unset unless intentional

CSV requirements:

- A header row with a `contact_number` column (E.164 format).
- Any other column becomes a prompt variable, matched by column name.

Response · same shape as §9.2(1), but **warnings is always empty** here:

```
{ "batch_id": "b_def456", "state": "created", "warnings": [] }
```

Note: the upload path does **not** run the required-variable validation that `POST /v1/batches` does · turing can inspect CSV rows the same way (so a missing required variable fails silently on the voice engine's side instead of a 422 up front). Prefer the JSON endpoint when you want the pre-flight check; use upload for bulk lists you've already validated on your side.

9.3 POST /v1/batches/{batch_id}/schedule

Delay a created batch to a future time instead of running immediately.

```
curl -X POST
https://turing.docstribе.com/v1/batches/b_abc123/schedule \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
  -d '{"scheduled_at": "2026-07-20T09:30:00+00:00",
    "bypass_call_guardrails": false}'
```

Field	Required	Notes
scheduled_at	yes	ISO 8601, e.g. 2026-07-20T09:30:00+00:00. A bare Z suffix is fine too · turing normalizes it to +00:00 for you before forwarding upstream. Must be at least 2 minutes in the future ; rounded up to the next 10-minute mark by the voice engine.
bypass_call_guardrails	no	Skips the voice engine's calling-time guardrail checks (e.g. do-not-call windows). Leave unset/false unless you know you need this.

```
{ "message": "Batch scheduled.", "state": "scheduled" }
```


9.4 GET /v1/batches/{batch_id} · status

```
curl https://turing.docstribе.com/v1/batches/b_abc123 -H
"X-API-Key: $KEY"
```

```
{
  "batch_id": "b_abc123",
  "status": "in_progress",
  "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
  "scheduled_at": null,
  "from_phone_numbers": ["+91XXXXXXXXXX"],
  "valid_contacts": 120,
  "total_contacts": 125,
  "created_at": "2026-07-12T18:44:54.410785Z",
  "updated_at": "2026-07-12T19:01:02.100000Z"
}
```

status is refreshed live from the voice engine while the batch is active; once terminal (completed/stopped/failed/deleted) it's served straight from turing's own record.

9.5 GET /v1/batches · list all your batches

```
curl "https://turing.docstribе.com/v1/batches?status=in_progress"
-H "X-API-Key: $KEY"
```

Returns up to **200** batches for your client, newest first. Optional query params: **agent_id** (filter by agent) and **status** (filter by batch status). There is no pagination · if you have more than 200 batches, only the most recent 200 are returned.

9.6 GET /v1/batches/by-agent/{agent_id} · list your batches for an agent

```
curl https://turing.docstribе.com/v1/batches/by-agent/37768781...
-H "X-API-Key: $KEY"
```

Optional query param: **limit** (default **200**, range 1-500).

```
[
  {
    "batch_id": "b_abc123",
    "internal_id": "5b0e9390-4d07-43b7-aba8-cfcf1099fe11",
    "status": "completed",
    "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
    "scheduled_at": null,
    "from_phone_numbers": ["+91XXXXXXXXXX"],
    "valid_contacts": 120,
    "total_contacts": 125,
    "created_at": "2026-07-12T18:44:54.410785Z",
    "updated_at": "2026-07-12T20:10:00.000000Z"
  }
]
```

Returns only **your** batches for that agent, newest first · this reads turing's own database, not the upstream engine, so it's fast and correctly scoped even though the upstream account is shared across all clients. **internal_id** is turing's own UUID for the batch (used by the frontend to link calls and analytics); **batch_id** is the voice engine's id.

9.7 GET /v1/batches/{batch_id}/executions · per-call results

```
curl https://turing.docstribе.com/v1/batches/b_abc123/executions -H
"X-API-Key: $KEY"
```

```
[
  {
    "id": "exec_xyz",
    "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
    "status": "completed",
    "conversation_duration": 42.5,
    "total_cost": 0.15,
    "transcript": "...",
    "extracted_data": {"outcome": "confirmed"},
    "telephony_data": {"to_number": "+919876543210",
    "recording_url": "..."}
  }
]
```

This is also your **reconcile path** · call it any time to pull the latest state for every call in the batch, independent of whether the webhook (§11) fired successfully. Safe to poll periodically (e.g. every few minutes while a batch is active) as a backstop.

9.8 GET /v1/batches/{batch_id}/metrics · aggregate stats

```
curl https://turing.docstribе.com/v1/batches/b_abc123/metrics -H
"X-API-Key: $KEY"
```

```
{
  "batch_id": "1234...-uuid",
  "voice_batch_id": "b_abc123",
  "status": "completed",
  "total_recipients": 125,
  "calls_tracked": 125,
  "by_status": {"completed": 100, "no-answer": 20, "failed": 5},
  "completed": 100,
  "terminal": 125,
  "success_rate": 0.8,
  "total_cost": 18.75,
  "avg_duration_seconds": 38.2
}
```

Computed from turing's own stored call records, not a live upstream call · cheap to call frequently.

9.9 POST /v1/batches/{batch_id}/stop

Halts a queued or running batch.

```
curl -X POST https://turing.docstribе.com/v1/batches/b_abc123/stop
-H "X-API-Key: $KEY"
```

9.10 DELETE /v1/batches/{batch_id}

Removes the batch on the voice engine. turing retains your historical record (status, metrics, call history) regardless.

```
curl -X DELETE https://turing.docstribе.com/v1/batches/b_abc123 -H
"X-API-Key: $KEY"
```

10. Endpoint summary

Method	Path	Auth	Purpose
POST	/v1/register	None	Register your client
GET	/v1/claim/{token}	None	Peek at claim link
POST	/v1/claim/{token}	None	Burn claim & get API key
POST	/v1/portal/lookup	None	Portal login via name+email
GET	/v1/me	Key	Your client profile
GET	/v1/me/config	Key	Your webhook & default config
GET	/v1/me/keys	Key	List your API keys
POST	/v1/me/keys	Key	Create a new API key
DELETE	/v1/me/keys/{key_id}	Key	Revoke an API key
GET	/v1/agents	Key	List agents enabled for you
GET	/v1/agents/{agent_id}/variables	Key	Required/optional prompt variables
GET	/v1/agents/drift	Key	Detect removed/missing agents
GET	/v1/phone-numbers	Key	Available caller IDs + your default
POST	/v1/calls	Key	Make a single call
GET	/v1/calls	Key	List calls (paginated)
GET	/v1/calls/{execution_id}	Key	Call details + transcript
POST	/v1/calls/{execution_id}/stop	Key	Stop in-flight call
POST	/v1/calls/{execution_id}/analyze	Key	Run LLM analysis on call
GET	/v1/batches	Key	List all your batches
POST	/v1/batches	Key	Create batch from JSON
POST	/v1/batches/upload	Key	Create batch from CSV
POST	/v1/batches/{batch_id}/schedule	Key	Schedule a created batch
GET	/v1/batches/{batch_id}	Key	Batch status
GET	/v1/batches/by-agent/{agent_id}	Key	Your batches for an agent
GET	/v1/batches/{batch_id}/executions	Key	Per-call results (reconcile)
GET	/v1/batches/{batch_id}/metrics	Key	Aggregate campaign stats
POST	/v1/batches/{batch_id}/stop	Key	Stop a running batch
DELETE	/v1/batches/{batch_id}	Key	Delete a batch
GET	/v1/analytics/overview	Key	Overall call stats
GET	/v1/analytics/by-agent	Key	Stats grouped by agent
GET	/v1/analytics/by-batch	Key	Stats grouped by batch
GET	/v1/analytics/timeseries	Key	Time-series call data
POST	/v1/sql-agent/query	Key	Natural-language SQL queries (optional)

11. Receiving outcomes · your webhook endpoint

Ask turing's operator to set your `webhook_url` (and a `webhook_secret`) on your client config. You can verify what's configured via `GET /v1/me/config` (§13). Once set, turing will `POST` a lean outcome to that URL every time a call in one of your batches reaches a new state · you don't need to poll (though §9.7 is there as a backstop).

Payload

```
POST <your webhook_url>
Content-Type: application/json
X-Webhook-Signature: sha256=<hex hmac>

{
  "turing_call_id": "5b0e9390-4d07-43b7-aba8-cfcf1099fe11",
  "turing_batch_id": "b_abc123",
  "voice_call_id": "exec_xyz",
  "patient_uhid": "test-patient-1",
  "contact_number": "+919876543210",
  "agent_id": "37768781-4fc4-4df2-a80f-a847b6dad8d2",
  "status": "completed",
  "disposition": null,
```

```

    "recording_url": "https://.../recording.mp3",
    "cost": 0.15,
    "duration": 42.5,
    "hangup_reason": "user_hangup"
}

```

turing_batch_id is the same batch_id you got from POST /v1/batches · use it to correlate the callback with your campaign.

Verifying the signature

The signature is an HMAC-SHA256 over the **exact raw request body**, hex digested, prefixed sha256=, keyed with the webhook_secret you were given. **Verify it before trusting the payload.**

Python:

```

import hashlib, hmac
def verify(raw_body: bytes, signature_header: str, secret: str) -> bool:
    expected = "sha256=" + hmac.new(secret.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature_header)

```

Node.js:

```

const crypto = require("crypto");
function verify(rawBody, signatureHeader, secret) {
    const expected = "sha256=" + crypto.createHmac("sha256",
secret).update(rawBody).digest("hex");
    return crypto.timingSafeEqual(Buffer.from(expected),
Buffer.from(signatureHeader));
}

```

Important: hash the **raw bytes** of the request body, not a re-serialized version of the parsed JSON (key ordering/whitespace differences will break the comparison).

Reliability

- Delivery is best-effort with **no retry at all** · a single attempt per event. If your endpoint is down, slow, or errors, turing logs it and moves on; that outcome update is not redelivered. Use GET .../executions) as a periodic reconcile poll so a missed webhook never means missed data · treat the webhook as a low-latency notification, not your source of truth.
- Respond 2xx quickly; do slow processing (e.g. writing to your own DB) after acknowledging, not before.
- Your endpoint should be idempotent on voice_call_id · the same execution may be delivered more than once as its status progresses (e.g. ongoing · completed).

12. Analytics

Query your call data across agents, batches, and time ranges.

GET /v1/analytics/overview

```
curl
```

```
"https://turing.docstribе.com/v1/analytics/overview?date_from=2026-08-01&date_to=2026-08-01"
-H "X-API-Key: $KEY"
```

Returns: total calls, completed, failed, total cost, average duration, etc. for the date range.

Query params: date_from, date_to, agent_id, batch_id (all optional).

GET /v1/analytics/by-agent

Same filters, but results grouped per agent.

GET /v1/analytics/by-batch

Same filters, but results grouped per batch.

GET /v1/analytics/timeseries

Returns data points over time. Additional param: granularity=day|week.

13. Self-serve management

Once authenticated, you can manage your own profile and API keys.

GET /v1/me · your client profile

```
curl https://turing.docstribе.com/v1/me -H "X-API-Key: $KEY"

{
  "client_id": "b3f1...-uuid",
  "name": "Your Company",
  "slug": "your-company",
  "contact_email": "eng@yourcompany.com",
  "status": "active",
  "created_at": "2026-08-01T...",
  "approved_at": "2026-08-01T...",
  "active_key_count": 2
}
```

GET /v1/me/config · your webhook & default config

```
curl https://turing.docstribе.com/v1/me/config -H "X-API-Key: $KEY"

{
  "default_from_number": "+91XXXXXXXXXX",
  "webhook_url":
    "https://your-backend.example.com/turing/callback",
  "webhook_secret_set": true,
  "visible_fields": null,
  "settings": null
}
```

webhook_secret_set tells you whether a signing secret is configured (the actual secret is never returned). If

you need to change any of these values, ask turing's operator · there is no self-service write endpoint.

GET /v1/me/keys · list your API keys

Returns all keys (prefix + label + status + last used). The raw key is never shown again after issuance.

POST /v1/me/keys · create a new API key (201)

```
curl -X POST https://turing.docstribе.com/v1/me/keys \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
  -d '{"label": "production-backend"}'
```

Returns the raw key once. Store it securely.

DELETE /v1/me/keys/{key_id} · revoke a key

Immediately invalidates the key. Cannot be undone.

14. Worked end-to-end flow

```
KEY="tk_your_api_key"
HOST="https://turing.docstribе.com"
# 1. Discover your enabled agents
curl $HOST/v1/agents -H "X-API-Key: $KEY"
# 2. Check what an agent needs per recipient
curl $HOST/v1/agents/<agent_id>/variables -H "X-API-Key: $KEY"
# 3. Place a batch
curl -X POST $HOST/v1/batches -H "X-API-Key: $KEY" -H
"Content-Type: application/json" \
  -d

'{"agent_id":"<agent_id>","recipients":[{"contact_number":"+91...", "<var>":"..."}]}'
# 4. Poll status (or just wait for your webhook)
curl $HOST/v1/batches/<batch_id> -H "X-API-Key: $KEY"
# 5. Pull per-call results / reconcile
curl $HOST/v1/batches/<batch_id>/executions -H "X-API-Key: $KEY"
# 6. Get aggregate metrics once done
curl $HOST/v1/batches/<batch_id>/metrics -H "X-API-Key: $KEY"
```

Meanwhile, your webhook endpoint receives a signed outcome POST for each call as it completes · no polling required for real-time updates, but §4 and §9.7 are there if you need them.

15. SQL Builder Agent (optional)

If enabled for your client, the SQL Builder Agent converts natural-language questions into validated PostgreSQL queries against a configured data source.

POST /v1/sql-agent/query

```
curl -X POST https://turing.docstribе.com/v1/sql-agent/query \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
```

```
-d '{"question": "How many patients visited last week?",
"workspace": "kalaam"}'
```

Request fields:

Field	Required	Notes
question	yes	Natural-language question
workspace	no	Target data source (defaults to "kalaam")

Response:

```
{
  "status": "success",
  "query": "SELECT COUNT(*) FROM patient_visits WHERE created_at >=
...",
  "reasoning": "Counted distinct patient visits in the last 7
days...",
  "result": [{"count": 1247}]
}
```

The agent validates generated SQL with EXPLAIN before execution and returns an error status if the question cannot be answered safely.